

Interfaces

Interface (Schnittstelle)

- beschreiben **Eigenschaften**, die erbende Klassen haben müssen
- gibt nur die Struktur von Eigenschaften und Methoden an
→ es definiert Member, ohne diese zu implementieren
- eine Klasse kann von einem oder mehreren Interfaces erben und muss alle Member implementieren
- es kann **kein Objekt** von einem Interface erstellt werden

- alle Member sind standardmäßig public und abstract
- Zugriffsmodifizierer können nicht explizit benutzt werden (kein private, protected oder internal)
- können keine Felder definieren (enthält keine Speicherung von Daten)

Interfaces beschreiben meistens Eigenschaften zum Beispiel Objekt ist „fahrbar“

```
// Schnittstelle IDriveable (etwas ist fahrbar)
interface IDriveable
{
    // Fahrbare Objekte müssen die Eigenschaften Name und Geschwindigkeit haben
    // Fahrbare Objekte können beschleunigen und bremsen

    // Definition einer Eigenschaft Name (lesbar & veränderbar)
    string Name { get; set; }

    // Definition einer Eigenschaft Geschwindigkeit (nur lesbar)
    double Speed { get; }
    // Definition einer Methode zum Beschleunigen
    void Accelerate(double deltaSpeed);

    // Definition einer Methode zum Bremsen
    void Brake(double deltaSpeed);
}
```

Klassen können auch mehrere Interfaces implementieren.

// Klasse Car implementiert Schnittstellen IDriveable (Auto ist fahrbar) und ILightable (Auto kann leuchten)

```
class Car : IDriveable , ILightable
```

```
{
```

```
    // Eigenschaften Name und Speed von IDriveable
```

```
    public string Name { get; set; }
```

```
    public double Speed { get; private set; } = 0; // nur veränderbar in der Car Klasse; Standardwert 0
```

```
    // Eigenschaft IsLightsOn von ILightable
```

```
    public bool IsLightsOn { get; set; }
```

```
    // Methode Accelerate und Break von IDriveable müssen implementiert werden
```

```
    public void Accelerate(double deltaSpeed)
```

```
    {
```

```
        Speed += deltaSpeed;
```

```
    }
```

```
    public void Brake(double deltaSpeed)
```

```
    {
```

```
        Speed -= deltaSpeed;
```

```
    }
```

```
}
```

Methoden und statische Properties

- Statische Member sind nur über das Interface aufrufbar
- Wenn sie eine Standartimplementierung besitzen dürfen überschrieben werden

```
interface IDriveable
{
    string Name { get; set; }
    double Speed { get; }

    // Statische Konstante für die maximale Geschwindigkeit
    static double MaxSpeed { get; set; } = 300;

    void Accelerate(double deltaSpeed);
    void Brake(double deltaSpeed);
}
```

```
class Car : IDriveable , ILightable
{
    public string Name { get; set; }
    public double Speed { get; private set; } = 0;
    public bool AreLightsOn { get; set; }

    // Methode Accelerate und Break von IDriveable
    public void Accelerate(double deltaSpeed)
    {
        // Statische Konstante von IDriveable wird verwendet
        if (Speed + deltaSpeed <= IDriveable.MaxSpeed)
        {
            Speed += deltaSpeed;
        } else
        {
            Speed = IDriveable.MaxSpeed;
        }
    }
    public void Brake(double deltaSpeed)
    {
        Speed -= deltaSpeed;
    }
}
```

implizierte Implementierung

- Zugriff auf Member direkt über den Objekttyp

```
// Klasse Program
class Program
{
    // Main Methode
    static void Main(string[] args)
    {
        // Neues Car-Objekt erstellen
        Car car = new Car();
        // Zugriff auf Accelerate-Methode von IDriveable über Car-Objekt
        car.Accelerate(10);
    }
}
```

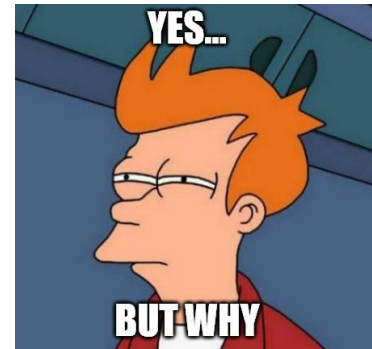
explizierte Implementierung

- die Member können keine Zugriffsmodifizierer verwenden
- Zugriff nur über Konvertierung in den Typ des Interfaces

```
class Car : IDriveable
{
    public string Name { get; set; }
    public double Speed { get; private set; } = 0;

    // Explizite Implementierung von IDriveable.Accelerate
    void IDriveable.Accelerate(double deltaSpeed)
    {
        Speed += deltaSpeed;
    }
}

Car car = new Car();
// Konvertierung von Car zu IDriveable
IDriveable driveableCar = (IDriveable) car;
driveableCar.Accelerate(10);
```



explizierte Implementierung: Beispiel FlyingCar

```
// (etwas ist fahrbar)
interface IDriveable
{
    string Name { get; set; }
    double Speed { get; }
    void Accelerate(double deltaSpeed);
    void Brake(double deltaSpeed);
}
```

```
// (etwas ist flugfähig)
interface IFlyable
{
    string Name { get; set; }
    double Altitude { get; }
    void Accelerate(double deltaSpeed);
    void Brake(double deltaSpeed);
}
```

explizierte Implementierung: Beispiel FlyingCar

```
// Klasse FlyingCar implementiert beide Schnittstellen
class FlyingCar : IDriveable, IFlyable
{
    public string Name { get; set; }
    public double Speed { get; private set; }
    public double Altitude { get; private set; }

    // Explizite Implementierung von IDriveable.Accelerate
    void IDriveable.Accelerate(double deltaSpeed)
    {
        Speed += deltaSpeed;
    }

    // Explizite Implementierung von IDriveable.Brake
    void IDriveable.Brake(double deltaSpeed)
    {
        Speed -= deltaSpeed;
    }

    // Explizite Implementierung von IFlyable.Accelerate
    void IFlyable.Accelerate(double deltaSpeed)
    {
        Altitude += deltaSpeed;
    }

    // Explizite Implementierung von IFlyable.Brake
    void IFlyable.Brake(double deltaSpeed)
    {
        Altitude -= deltaSpeed;
    }
}
```

```
static void Main()
{
    // Neues Objekt flyingCar
    FlyingCar flyingCar = new FlyingCar();

    // Casten von flyingCar zu IDriveable und IFlyable
    IDriveable driveable = (IDriveable) flyingCar;
    IFlyable flyable = (IFlyable) flyingCar;

    // die gleichnamigen Methoden können jetzt aufgerufen werden
    driveable.Accelerate(100);
    flyable.Accelerate(100);
}
```